

Sponsor Company: Optimization of LiteRT Micro for RISC-V

(Phase 2 - with ECE team)

Prepared by Alec Freeman, Ollie Rayo, Avinash Mandala, James Dong

University of Texas at Dallas

ECE Team: James Whitfield, Akhilesh Vemuri, Jack Liao

Company/Sponsor: Peanut Micro LLC

Asheesh Kashyap - Company Mentor

Dr. Beiyu Lin - CS UT Dallas Faculty Advisor

12/9/2025

Abstract:

Speed while maintaining low power consumption is one of the key requirements of Tiny Machine Learning (TinyML) applications. However, many TinyML applications rely on scalar microcontrollers or interact with the cloud, slowing down performance. As a result, Peanut Micro LLC aims to implement optimized kernels for a RISC-V vector processor that are faster and more power-efficient. To do this, we identify performance-critical operations, mainly of person detection and speech recognition, in TensorFlow Lite for Microcontrollers (LiteRT Micro), implement optimizations of these operations using RISC-V vector instructions, and use a developed cycle-level performance analyzer to quantify performance. Our optimized kernels achieved over a 8-10x speedup at the operator level and more than a 4x improvement in application-level speedups over the base scalar implementation. By open-sourcing these kernels, we enhance the RISC-V software ecosystem and demonstrate the performance and efficiency benefits of vector extensions in TinyML applications.

Table of Contents:

INTRODUCTION	3
TIMETABLE	5
MAIN MILESTONES	5
PROJECT METRICS	6
KEY ROLES	8
COMMUNICATION PLAN	9
RISK ANALYSIS/CONTINGENCY PLAN:	10
EVALUATION/TRACKING PLAN	11
ETHICS DISCUSSION	12
IMPACT, SECURITY, AND PRIVACY	13
10.1 Impact/Benefits	13
10.2 Security/Privacy	13
IMPLEMENTATION DETAILS	14
11.1 Performance Analysis Script	14
11.2 Optimized Kernels	14
11.3 Performance Analysis Process	15
INDIVIDUAL ASSESSMENT	16
ISSUES AND LESSONS LEARNED	17
FUTURE WORK	18
MEETING MINUTES	19
15.1 Meeting Minutes 1: 9/12	19
15.2 Meeting Minutes 2: 9/19	20
15.3 Meeting minutes 3: 9/26	23
15.4 Meeting minutes 4: 10/3	25
15.5 Meeting Minutes 5: 10/10	26
15.6 Meeting Minutes 6: (10/17)	29
15.7 Meeting Minutes 7: 10/24	30
15.8 Meeting Minutes 8: 10/31	32
15.9 Meeting Minutes 9: 11/7	33
15.10 Meeting Minutes 10: 11/14	34
15.11 Meeting Minutes 11: 11/21	35
15.12 Meeting Minutes 12: 11/28	36
15.13 Meeting Minutes 13: 12/5	36
SIGNATURES	38

INTRODUCTION

The objective of this project is to optimize LiteRT Micro for RISC-V microcontrollers by reducing the instruction count using intrinsic libraries targeting the RISC-V Vector Extension (RVV). Unlike previous projects and efforts that mostly focus on developing new hardware accelerators or custom ISA extensions from scratch, our approach is purely software-level optimization of TensorFlow Lite Micro or LiteRT Micro for TinyML applications. This makes the optimizations made directly applicable to already-existing RISC-V cores without having to make additional hardware modifications. There are two similar projects that have inspired ours.

The first, by a group of students from Southampton university, developed a custom RISC-V vector ISA extension for the CV32E40P RISC-V core and examined the speedup on neural network benchmarks. They found that they only had to modify 17 RISC-V vector instructions to reach this performance. Their work required hardware modifications and a fully custom ISA extension.

The second project, by Boston University, developed ISA extensions and in-pipeline execution units to accelerate neural network inference on RISC-V. They then integrated these instructions into LiteRT. The solution achieved a reduction of up to 8 times in executed instructions. It relied on modifying hardware along with software by using custom pipeline units that increase complexity and limit portability.

In contrast with the Southampton project, our project focuses on intrinsic-level optimization within LiteRT Micro for existing RISC-V cores with RVV. It requires no hardware changes while the Southampton project required hardware modifications. If hardware modifications are required, the instructions only work for the specific custom core they developed. Because our project is only modifying pre-existing RVV instructions by editing intrinsic functions, our project provides immediate applicability to any RISC-V system with RVV and is easier to maintain. We are reducing instruction count and hence increasing performance without having to build hardware.

The Boston University project requires custom pipeline units which increase design complexity and limit portability to other RISC-V cores. The software optimization in the project depended heavily on hardware support. However, our project will optimize existing vector intrinsics, only changing software and avoiding the overhead custom hardware units.

Current vendors for TinyML have all implemented optimized kernels or functions for essential TinyML applications, but these haven't been done using the RISC-V vector intrinsics. Since RISC-V is open-source, our intrinsic library of kernels that provide speed up to TinyML applications can be taken by anyone and run on their machine.

Overall, our project emphasizes portability, maintainability and practicality. By targeting intrinsic libraries we provide an immediately deployable solution for improving AI

performance for TinyML on RISC-V microcontrollers by modifying the intrinsic libraries that use RVV instructions.

TIMETABLE

MAIN MILESTONES

Phase 1: due 10/16 - Understand all educational material

- Description: Review LiteRT Micro, Tensor Flow Lite Micro tutorials, TinyML tutorials, RISC-V Vector Extension reference, understand vector intrinsic libraries and other relevant material to understand the background materials necessary to start the project

Phase 2 due 10/23 - Start project ramp up

- Description: Have the development environment set up, LiteRT micro cross-compiled for RISC-V, and analyze the kernels (for example, Convolution operations kernel) that are critical for optimization.

Sprint 1: due 11/6

- Description: Examine existing intrinsics used by the prioritized kernels, identify inefficiencies of certain intrinsic functions.

Sprint 2: due 11/20

- Description: Rewrite portions of LiteRT Micro's kernels such as the Add, Multiply, Pooling kernels and observe optimization improvement. Also rewrite portions of LiteRT Micro signal detection kernels such as the RFFT and FilterBank kernels.

Sprint 3: 11/27

- Description: Evaluate optimized kernels using the performance profiling script for speech recognition and person detection application benchmarks. Get the returned cycles for the vectorized intrinsic, autovectorized, and scalar optimization levels and execution modes.

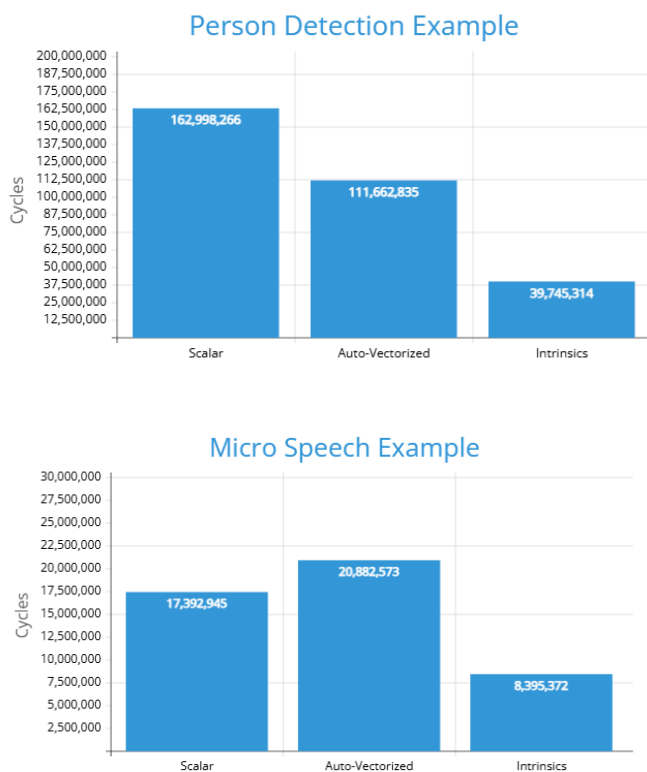
Phase 3 due 12/01 - Achieve further optimization and integrate kernels

- Description: Achieve further optimization of the intrinsic vector functions or kernels by rewriting the functions so they use less RVV instructions.
- Integrate optimized kernels or intrinsic functions and remove functions that don't provide speedup.
- Get results for intrinsics, autovectorized, and scalar kernel implementations

PROJECT METRICS

We used a profiler that was previously developed last semester to approximate the number of cycle counts it would take to run a vectorized assembly file within a reasonable level of accuracy. We estimate program performance by comparing the runtimes of the sample programs in the spike simulator. The simulator is run on a single threaded Ubuntu processor. We compared three implementations of the different application benchmarks and kernels: scalar, auto vectorization, and vector intrinsics. Scalar is the default implementation, running on a scalar or standard RISC-V processor with no vectorization applied. It represents the performance we are trying to improve upon. Auto vectorization doesn't use any of our custom vector intrinsic functions and is the compiler attempting to vectorize inefficient instructions on its own by detecting vectorizable patterns and generating vector instructions automatically. Finally, vector intrinsics uses our custom function implementations that explicitly call RISC-V vector intrinsic functions, allowing us to manually control vectorization. We evaluated these different kernel code implementations on two standard TinyML applications or benchmarks: person detection and micro speech (speech recognition). Both simply tell whether there is a certain image or word present respectively.

The results are as follows:



Using vector intrinsics, our custom kernels created massive speedups in our critical functions. Our profiling script demonstrated a 8-10x speedup compared to the scalar version of the operations for these critical operations, a 4x speedup for Person Detection, and 2.1x speedup for Speech Recognition. Auto-intrinsics actually fared worse than the scalar or baseline implementation for

Speech Recognition and we achieved more than a 2.5x speedup compared to the auto-intrinsics by using vector intrinsics.

KEY ROLES

All CS student team members (Alec Freeman, Ollie Rayo, Avinash Mandala, and James Dong) will share joint responsibility for all technical and project tasks, including analyzing LiteRT Micro kernels, identifying inefficiencies in intrinsic vector functions, optimizing intrinsic libraries using RVV, collecting performance metrics of the kernels. The CS team members are also responsible for their own assigned kernels: 8bit Add, Relu, and AvgPool.

- Notetaker: Students will alternate being the notetaker each week and being responsible for the Meeting Minutes for that week.
- Primary point of contact with mentor: Ollie Rayo
 - Will consolidate feedback and communicate progress in project

Team Leader - James Whitfield

- Responsible for assigning bottleneck kernels to team members
- Responsible for integrating the implemented kernels produced by each team member

ECE Team - James Whitfield, Akhilesh Vemuri, Jack Liao

- Role: Provide technical guidance to the team as they have previously worked on the project
- Implement their assigned kernels: MaxPool, Reshape, and 8-bit Multiply

CS advisor - Dr. Beiyu Lin

- Role: Provides academic guidance and advises on methodology and technical correctness

Mentor - Asheesh Kashyap

- Role: Provides domain expertise and project guidance. Also reviews the project direction, gives feedback on optimizations, and can help in assisting with technical challenges.

COMMUNICATION PLAN

- Interaction between teammates will occur through Discord.
- Interaction with the company mentor and faculty advisor will be either through the weekly meetings or through email.
- Meetings are always on Friday from 2-3pm unless the company mentor specifies a different time
- Attendees are kept track of by seeing who attends the meetings and then recording who and who does not attend in the weekly Meeting Minutes.
- Meeting Schedule
 - Every Friday from 2-3pm

RISK ANALYSIS/CONTINGENCY PLAN:

- Team Member Unavailability (Sick or MIA)
 - Risk: A key team member may be unavailable, delaying tasks.
 - Mitigation: All technical tasks are shared among team members, so the tasks are ensured to be complete but there is not too much assigned to one person. Key modifications will be maintained in a shared repository to ensure continuity.
- Timeline Slippage
 - Risk: Tasks take longer than expected, delaying milestones.
 - Mitigation: Inform mentor and teammates. Have another teammate help out. Apply sprint-based planning, monitor progress weekly, and adjust task allocation as needed. Critical tasks are prioritized to minimize overall schedule risk.
- Optimization or Debugging Challenges
 - Risk: Optimized kernels may produce incorrect results or fail to improve performance.
 - Mitigation: Implement unit tests or create scripts for correctness verification. Ask teammates or mentors for help. Allocate iterative sprints for refinement to ensure accuracy.

EVALUATION/TRACKING PLAN

- **End of Semester Evaluation**
 - At the end of the semester, the amount of speed up and the mentor's opinion on the speedup will determine how successful the project was.
 - If the speedup was similar to the previous semester or the mentor thinks that the speedup is successful, then the project is a success. Otherwise it is not.
- **Task Timing**
 - Every week the mentor will assign a task to do. If a task is completed prior to the next meeting then the task is completed on time.
 - More important tasks are established ahead of time and must be completed before their established due date in the proposal or the date specified by the mentor.
- **Progress Tracking**
 - The project will follow an iterative sprint-based approach which has clearly defined tasks for kernel analysis and intrinsic optimization.
 - Progress will be monitored using agile burn-down charts that estimate remaining work vs time.
- **Defect Tracking**
 - All code changes are maintained in a Git repository and peer reviews of optimizations will ensure correctness and removal of defects in the intrinsic functions.
 - Any failures in benchmark results should be noted, prioritized and dealt with before proceeding to the succeeding sprints.
- **Test plans**
 - Optimized kernels will be tested against baseline implementations to ensure outputs remain correct
 - Performance scripts will automatically measure runtime to measure performance of the kernels.

ETHICS DISCUSSION

- **Code**
 - The code used in this project will be open-source
 - No code will be borrowed or copied from non-open-source sources
- **Work Splitting**
 - Decisions on how work is split among team members will be made collectively with open discussion through the Discord
 - Conflicts or disagreements will also be solved in the Discord
 - Roles will be shared to ensure all members contribute meaningfully
- **Ethics of Project**
 - The project aims to optimize AI performance on embedded systems for research purposes, not for unethical applications.
 - We will optimize kernels that cannot be used for causing any type of harm

IMPACT, SECURITY, AND PRIVACY

10.1 Impact/Benefits

Overall, this project contributes to improving runtime performance of optimized kernels in LiteRT Micro for open source RISC-V processors which ultimately enables faster and more efficient AI applications on embedded devices and microcontrollers. Users of embedded devices that use AI will experience better responsiveness and efficiency, making AI on embedded devices more practical. The speedups result in any applications that use these functions to be faster and consume less power, including the application benchmarks used in the project, person detection and speech recognition.

Because the RISC-V instruction set architecture (ISA) is open source, anyone can use these intrinsic functions implemented in this project and run it on their machine. The software optimizations produced by the project could potentially be applied by companies or researchers to reduce development effort while improving the performance of AI-powered edge devices on RISC-V-based platforms. This is unlike optimizations made by current vendors for TinyML who have implemented functions, but not using RISC-V vector libraries or the RISC-V ISA, meaning that these changes cannot be used by the general public or by other companies. Optimizing kernels with vector intrinsics on RISC-V allows researchers or companies to expand our optimizations without proprietary limitations.

The project also could demonstrate the benefits of targeted intrinsic-level optimization. By targeting our optimization toward major kernels or functions, for signal detection, and convolution and fully connected kernels, for all TinyML applications, we achieve a tremendous speedup. These kernels originally served as a bottleneck on performance, but due to their high arithmetic intensity and regular data access patterns, provided the greatest performance benefit. This speedup is significantly greater than if we just optimized simple kernels such as Add or Multiply.

This supports the growth and use of the open-source RISC-V ecosystem and promotes standardized optimization practices to optimize embedded AI. Because of the significant speedups achieved, RISC-V vector extensions provide evidence that RISC-V vector processors can outperform scalar microcontrollers in TinyML workloads.

By improving the efficiency of AI on microcontrollers, the project also promotes energy-efficient computing. Optimizations make AI in small edge computers more practical, allowing these edge devices to be adopted in consumer devices, industrial automation and IoT applications.

10.2 Security/Privacy

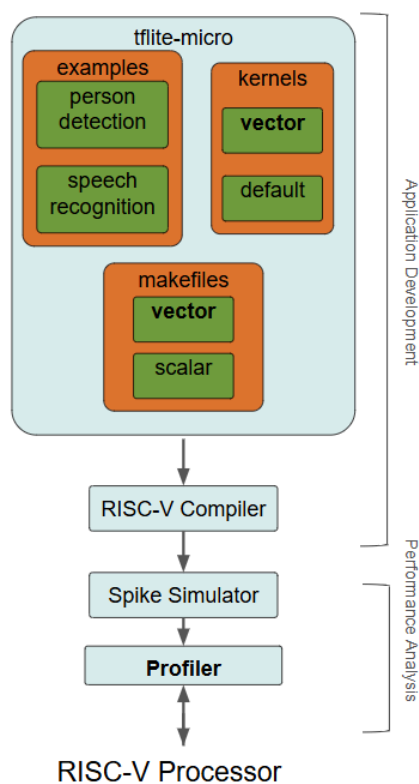
No personal data is used in this project. All resources, libraries, and toolchains are open-source, so the project doesn't have any confidentiality, availability, or integrity constraints. Any code or personal data in the shared code repositories are private to the team.

IMPLEMENTATION DETAILS

11.1 Performance Analysis Script

A critical part of this project, implemented last semester, for identifying problems and measuring performance improvements is the performance analysis script. Given an execution trace of a RISC-V program and a machine configuration (like rv32imcv or rv32imc), it goes through every instruction and returns the theoretical runtime of the program. In its current state, the machine configuration contains a mapping of [instruction]:[num cycles] with some further encoding for vector instructions. The total runtime for a benchmark application is given by listing the total cycles of the program trace produced by the spike simulator when it runs the associated application.

The general architecture is shown below:



11.2 Optimized Kernels

Many TinyML applications are developed using tflite-micro, which defines many common machine learning operations or kernels. The purpose of this project was to implement our own version of these functions using RISC-V vector intrinsics which reduce cycle counts by allowing the use of vector hardware instead of slow scalar instructions, and the emittance of the exact vector instructions instead of those assigned by a compiler in auto-vectorization. Then we can swap out the kernels or operations in tflite-micro for the implementations we designed.

Using the C++ RISC-V intrinsics found at https://github.com/riscv-non-isa/rvv-intrinsic-doc/tree/v0.11.x/auto-generated/intrinsic_funcs, we developed our own vectorized versions of tflite-micro kernels along with the ones implemented last semester. Altogether, these included primarily depthwise convolution, fully connected layer evaluation, softmax, RFFT, Filter Bank, and Pooling kernels.

Within our implementation of tflite-micro, our kernels are located in the tflite-micro/tensorflow/lite/micro/kernels/riscv_vector folder, and are generally over 100 lines long and are linked below:

[tflite-micro/tensorflow/lite/micro/kernels/riscv_vector/signal at main · Peanut-Microsystems/tflite-micro](#)

We also had to adapt the existing TensorFlow Lite Micro makefile by adding support for our vectorized kernel implementations. This allowed our optimized kernels to be integrated with the existing TFLite build system. To ensure a fair comparison we compiled the scalar, autovectorized, and vector intrinsic versions of the kernels using the same exact build configuration and toolchain options.

11.3 Performance Analysis Process

Many TinyML applications run over many iterations and require startup code in the setup phase of the application. This setup phase can skew the results of the performance of the applications. As a result we evaluated the performance by determining the performance of an application running for 1 iteration and then for 2 iterations and then taking the difference of the two to get the single-iteration latency. This single-iteration latency is used to compare the three versions.

INDIVIDUAL ASSESSMENT

Alec Freeman-Documentor and Implementor: My primary contributions were documentation and the implementation of the 8-bit add kernel and my own version of the AvgPool kernel. I was in charge of designing and writing the project proposal, designing the slide, poster, speech, and final proposal for the group, primarily contributing the content rather than the formatting. I also was responsible for the design of the 8-bit Add kernel. My contributed implementation actually provided no effect at all at the application level and actually provided a slowdown for the operation as a whole. I continually tested, received feedback, and continually optimized the kernel until I came to the conclusion that the Add kernel optimization will have little effect on the kernel and application performance. I also helped implement the AvgPool kernel, which also provided little to no speedup.

Completed Tasks:

10/10 - Project Proposal

11/20 - 8-bit Add kernel Finished

11/27 - Finished own implementation of AvgPool

11/30 - Designed Slide, Poster, Speech

12/9 - Final proposal

ISSUES AND LESSONS LEARNED

Alec Freeman: The primary issue I faced was the inability to provide speedup for simple operations such as Add and AvgPool. During the previous semester, the depthwise convolution and fully connected layer evaluation kernels were implemented, providing the majority of speedup for TinyML applications. However, this semester we were assigned simple kernels such as Add and AvgPool. However, I observed that despite my best attempts to implement these kernels, no speedup was being achieved. To solve this, I continually tested and implemented more optimized versions of Add and BroadcastAdd and evaluated their effects. I also asked for feedback from the ECE team who had previously worked on the project last semester, but they could also not find a reason why my kernels were not achieving the ideal speedup. As a result I moved on to AvgPool where I learned that unlike the signal detection and convolution kernels that were implemented, simple kernels such as Add and AvgPool are simply not worth implementing because their operation complexity is too simple. In these simple functions, performance is dominated by memory access rather than computation due to their low arithmetic performance. From this I've learned that it is important to prioritize highly compute-intensive kernels and to move on quickly to other kernels when not achieving any progress.

Two major concepts I used from my previous courses are debugging strategies and the use of registers. Effective debugging techniques I've developed from working on several projects for Computer Science courses (I and II specifically) have helped me isolate issues in the build system and with my kernels, validate correctness across scalar and vectorized implementations, and ensure that kernel outputs matched the baseline. My understanding of registers in Computer Architecture provided the foundation for understanding vector registers that are needed to write and optimize RISC-V vector intrinsic code.

One major thing I learned during this term to succeed during this project was how to use RISC-V vector intrinsics to exploit parallelism. Understanding vector registers, memory alignment and the instruction formats were crucial in the implementation of the vector intrinsics and reading other group members' code. In general, I've learned the process of optimization and the importance of moving on from optimizations that provide little to no speedup.

FUTURE WORK

Beyond the optimizations we made, to improve the project's outcome, additional kernels can be optimized such as those for signal detection(such as IIR or FIR filters), other activation functions beside ReLU, and Normalization. The tremendous speedup achieved by the kernels on the benchmark applications could be used in collaboration with a RISC-V vector processor team to integrate our optimized kernels directly with their hardware. This way, performance could be measured on FPGA hardware to validate our cycle-level predictions and highlight further opportunities for optimization.

Additionally, benchmarking could be expanded to include additional workloads such as image classification, keyword spotting, and sensor fusion, demonstrating broader applicability of vectorized kernels.

Finally, because this is an open-source contribution to the open-source RISC-V, the development of document tutorials and test suites for optimized kernels could make it easier for the community to adopt and extend them, so they can develop new RISC-V TinyML applications.

MEETING MINUTES

15.1 Meeting Minutes 1: 9/12

Project Title: Optimization of LiteRT Micro for RISC-V (Phase 2 - with ECE team)

Date: September 12, 2025

Time: 2pm

Names with Roles:

- Alec Freeman - CS Team Member
 - Avinash Mandala - CS Team Member
 - James Dong - CS Team Member
 - Ollie Rayo - CS Team Member
-
- Asheesh Kashyap - Company Mentor
 - Dr. Beiyu Lin - CS UT Dallas Faculty Advisor
-
- James Whitfield - ECE Veteran Team Member
 - Akhilesh Vemuri - ECE Veteran Team Member
 - Jack Liao - ECE Veteran Team Member

Absent participants:

- No absent members

Note taker:

- Ollie Rayo

Meeting Agenda

- Introduced the GitHub Resources
- Discussed what was done in the previous semester for the project
 - Developed a performance estimator tool to estimate the cycles for running assembly code with scalar processors, auto-vectorizing compilers, and intrinsic libraries, where intrinsic libraries gave impressive speedup for even complex tasks.
 - Optimized micro-kernels at application level
- Discussed what was to be done this semester

- Further optimizing the application level kernels that have already obtained great speedups from the previous semester of the project
- For the week after -> have to install toolchain and TinyML and set up applications.
- Introduced the magic of risc-v vector processing on what vector processing is
 - vector processor playlist for more

Activities agreed upon:

- Viewing educational materials for background knowledge
 - (Don't have to worry about VITUS resources)
- Watching RISC-V Videos
 - Explaining RISC-V video
 - RISC-V Assembly video
- Watching TinyML Videos
- Watching Vector Processing Videos
 - The Magic of RISC-V Vector Processing
- Reading Research Papers
 - Towards Deep Learning Using Tensorflow Lite on RISC-V

Data and time of the next meeting

- September 19 at 2pm

Name of person taking notes for next meeting

- Alec Freeman

Deadlines to submit the agenda for the next meeting

- 9/19/2025

Other Notes:

There was no meeting Friday September 5th as the sponsor did not have time to schedule one.

15.2 Meeting Minutes 2: 9/19

Project Title: Optimization of LiteRT Micro for RISC-V (Phase 2 - with ECE team)

Date: September 19, 2025

Time: 2pm - 3pm

Names with Roles:

- Alec Freeman - CS Team Member
 - Avinash Mandala - CS Team Member
 - James Dong - CS Team Member
 - Ollie Rayo - CS Team Member
-
- Asheesh Kashyap - Company Mentor
 - Dr. Beiyu Lin - CS UT Dallas Faculty Advisor
-
- James Whitfield - ECE Veteran Team Member
 - Akhilesh Vemuri - ECE Veteran Team Member
 - Jack Liao - ECE Veteran Team Member

Absent participants:

- No absent members

Note taker:

- Alec Freeman

Meeting Agenda

- Stated that the semester is flexible but following the steps of last semester will be beneficial
- Discussed the toolchain to be downloaded
 - The toolchain downloaded this week, the scalar version, is not the actual version we will be using
 - The vector version will be the actual version used which we are not downloading this week.
 - Downloading the toolchain:
 - In toolchain repository
 - Download windows subsystem from linux
 - Setting up git for within the virtual machine or WSL-Doesn't always work
 - Install RISC-V toolchain
 - Started with scalar instruction set last semester
 - Migrated to vector instruction set
 - Try and get base/scalar instruction set

- If you want to install the precompiled toolchain from GoogleDrive email Ash with your email, so you can get the toolchain shared through GoogleDrive.
- Or can build the toolchain from source to get latest
- Discussed Tensor Flow Lite
 - LiteRT = TFL
 - Version of tensor flow that runs on microcontrollers
 - Run executable as a RISC-V program
- Recommended Educational materials
 - Explaining RISC-V
 - Vector Processing
 - Magic of RISC-V Vector Processing
 - Introduction to the RISC-V vector processing from Barcelona Supercomputing center
- Discussed what was done last semester
 - Started with scalar instruction set
 - Migrated to vector instruction set
- Discussed how to optimize
 - Focus on speed, cycle reduction
 - Don't focus on memory

Activities agreed upon:

Mainly:

- Setup RISC-V toolchain with scalar processor
- Watch video on Magic of Vector Processing
- Carefully look at RISC-V Vector Tutorial from Barcelona Supercomputing center

Also:

- Learn about RISC-V instruction sets
- Take a peak at vector extensions

Data and time of the next meeting

- September 26 at 2pm

Name of person taking notes for next meeting

- Avinash

Deadlines to submit the agenda for the next meeting

- 9/26/2025

15.3 Meeting minutes 3: 9/26

Project Title: Optimization of LiteRT Micro for RISC-V (Phase 2 - with ECE team)

Date: September 26, 2025

Time: 2:15 pm - 3:15 pm

Names with Roles:

- Alec Freeman - CS Team Member
- Avinash Mandala - CS Team Member
- James Dong - CS Team Member
- Ollie Rayo - CS Team Member

- Asheesh Kashyap - Company Mentor
- Dr. Beiyu Lin - CS UT Dallas Faculty Advisor

- James Whitfield - ECE Veteran Team Member
- Akhilesh Vemuri - ECE Veteran Team Member
- Jack Liao - ECE Veteran Team Member

Absent participants:

- No absent members

Note taker:

- Avinash Mandala

Meeting Agenda

- Firstly made sure all tasks from last meeting that were assigned were done with no issues or blockers
 - We didn't have access to the google drive so we couldn't full setup the RISC-V toolchain
 - We were given a way to access the google drive and the prebuilt toolchain
- We got access to the peanut microsystems google drive
- Barcelona Supercomputer Center - read the slides
- Advanced_configurations.md
 - We are using the vector extension, not the full one but a subset
 - rv32imc_zve32x_zvl128b
 - 432 or 816 bits at a time

- Setup Tensorflow LiteRT Micro for TinyML applications
- TinyML
 - How to run an AI on a microcontroller
 - Memory constraints
 - Processing constraints
 - Size and Power constraints
 - Utilize the power of AI to enhance sensor data or endpoint data processing
- We won't be making our own LiteRT models but instead hacking it and modifying it to make the processor run faster
- Went over RISC-V Vector Tutorial from Barcelona Supercomputing center
 - Either 32 bit or 64 bit
 - Mainly focussing on Zv^* Vectors (Vector computation)
 - Could be 2 elements of 64 bits or 4 elements of 16 bits
 - Two registers used when operating vectors in RVV
 - `vtype` : vector type
 - `vl` : vector length
 - Length multiplier - `lmul` register
 - A fraction of how many elements of a vector register you want
 - Standard element width - `sew` register

Activities agreed upon:

Mainly:

- Revisit RISC-V Vector Tutorial from Barcelona Supercomputing center
- Read TinyML PDF by Pete Warden, chapters 1-4
- Setup Tensorflow LiteRT Micro for TinyML applications
- Focus on Tensorflow LiteRT and not Vector Toolchain

Data and time of the next meeting

- October 3 at 2pm

Name of person taking notes for next meeting

- Avinash Mandala

Deadlines to submit the agenda for the next meeting

- 10/03/2025

15.4 Meeting minutes 4: 10/3

Project Title: Optimization of LiteRT Micro for RISC-V (Phase 2 - with ECE team)

Date: October 3, 2025

Time: 2:00 pm - 2:30 pm

Names with Roles:

- Alec Freeman - CS Team Member
- Avinash Mandala - CS Team Member
- James Dong - CS Team Member
- Ollie Rayo - CS Team Member

- Asheesh Kashyap - Company Mentor
- Dr. Beiyu Lin - CS UT Dallas Faculty Advisor

- James Whitfield - ECE Veteran Team Member
- Akhilesh Vemuri - ECE Veteran Team Member
- Jack Liao - ECE Veteran Team Member

Absent participants:

- Dr. Beiyu Lin - CS UT Dallas Faculty Advisor

Note taker:

- Avinash Mandala

Meeting Agenda

- We previously looked at scalar toolchains but now we want to focus on vector toolchains
- Vector intrinsic function
 - Implement these into our toolchain
- We previously built the scalar toolchain but now we can build the vector toolchain variation
- Last year: made an optimized version of person detection using toolchain and developed risc-v performance script that estimates performance
- How to optimize AI kernels using vector concepts
- This course won't focus on building and deploying AI models, but instead optimizing the machinery that runs the AI models

- Programming in Intrinsic at the program level itself
- Reducing the number of instructions being processed
- Focusing on instruction count not CPI
 - $(CPI * INS \text{ count} * \text{cycle time})$
- Next week - key kernel (in TinyML) : depth wise convolution

Activities agreed upon:

Mainly:

- Build riscv vector toolchain (riscv_toolchain_build.md)
 - Either build it or download a compiled version from the google drive
- Run person detection using scalar and vector toolchain and turn on auto compile options
- Take a look at the vector intrinsics library
 - Understanding their structure and how they are used

Data and time of the next meeting

- October 10 at 2pm

Name of person taking notes for next meeting

- Alec Freeman

Deadlines to submit the agenda for the next meeting

- 10/10/2025

15.5 Meeting Minutes 5: 10/10

Project Title: Optimization of LiteRT Micro for RISC-V (Phase 2 - with ECE team)

Date: October 10, 2025

Time: 2:00 pm - 3:00 pm

Names with Roles:

- Alec Freeman - CS Team Member
- Avinash Mandala - CS Team Member
- James Dong - CS Team Member
- Ollie Rayo - CS Team Member

- Asheesh Kashyap - Company Mentor
- Dr. Beiyu Lin - CS UT Dallas Faculty Advisor

- James Whitfield - ECE Veteran Team Member
- Akhilesh Vemuri - ECE Veteran Team Member
- Jack Liao - ECE Veteran Team Member

Absentees:

- No absent members

Note taker:

- Alec Freeman

Meeting Agenda

- Where to find resources in Github
 - Page that explains the repository
 - Risc32_vector_make_file_inc
- Repository for doing testing on person detection
 - Tflite-micro
 - Worked on last semester
 - Contain optimized versions of libraries
 - PEANUT-README.md
 - Instructions on how to use vector intrinsics
- Last semester: used human detection and did optimized kernels on it
 - Have depth wise convolution
 - tensorflow/lite/micro/kernels/riscv_vector/conv_rvv
 - Look at this implementation and compare with base one
 - Duplicate results CS students had using vector intrinsics to implement kernels
 - See speedup at application level
- [Final Report - Peanut Micro LLC: Optimization of LiteRC Micro - Google Docs](#)
- This week:
 - Recreate results from last semester
 - Have depth wise convolution
 - tensorflow/lite/micro/kernels/riscv_vector/conv_rvv
 - Look at this implementation and compare with base one
 - Run performance script on person detection to replicate results from previous semester
 - Follow instructions on LiteRT_Vector-Intrinsic Flow page
 - Start with hello_world example
 - See if getting difference between scalar and vector version
 - Try on person detection and speech recognition baseline versions
- Graphs:
 - https://docs.google.com/presentation/d/1CvC-7p-CD8rC6JdKOOOpAGnIHFGBKaNIi/edit?usp=drive_link&oid=116106366183282204325&rtpof=true&sd=true

- https://docs.google.com/presentation/d/1cL2QHwFwxUj2SgEf8-Xmd4aWE7pJwYuQecH_LxUxwdM/edit?usp=drive_link
- Last semester: Took person detection and speech recognition
 - Ran application in scalar mode and then auto-vectorizing compiler which doesn't do a good job
 - Auto-vectorizing compilers cannot be trusted in this toolchain to auto-vectorize. Is worse than the scalar
 - When run same application with intrinsic functions, we get a much better improvement
 - 3x speedup at application level - remarkable
 - Have examples and kernels and run through RISC-V compiler, spike simulator and performance analysis tool
- Performance tool
 - LiteRT-Vector-Intrinsic-Flow Github page
 - Goes through entire flow of how to build and test application
 - Can disassemble executable
 - Performance Script
 - Need execution trace
 - Run performance script on execution trace
 - Stats on total instructions executed, what type of instructions were executed
- Direction of project
 - Vector optimized versions of signal processing applications
- ECE students focus on signal processing functions
- CS students continue to look for optimizations in other applications/AI related kernels
- Softmax is bugged so try working on that

Activities agreed upon:

- Review depthwise convolution (conv_rvv) implementations and compare it to the base version

As a group:

- Follow instructions on LiteRT_Vector-Intrinsic Flow page
 - Run hello_world, person detection, and speech recognition using both toolchains
- Recreate all of the performance improvements from last semester and have a group presentation outlining the results and data (Essentially fully catch up to where the team ended last semester)

Data and time of the next meeting

- October 17 at 2pm

Name of person taking notes for next meeting

- Ollie Rayo

Deadlines to submit the agenda for the next meeting

- 10/17/2025

15.6 Meeting Minutes 6: (10/17)

Project Title: Optimization of LiteRT Micro for RISC-V (Phase 2 - with ECE team)

Date: October 17, 2025

Time: 2:00 pm - 3:00 pm

Names with Roles:

- Alec Freeman - CS Team Member
- Avinash Mandala - CS Team Member
- James Dong - CS Team Member
- Ollie Rayo - CS Team Member

- Asheesh Kashyap - Company Mentor
- Dr. Beiyu Lin - CS UT Dallas Faculty Advisor

- James Whitfield - ECE Veteran Team Member
- Akhilesh Vemuri - ECE Veteran Team Member
- Jack Liao - ECE Veteran Team Member

Absentees:

- Akhilesh Vemuri - ECE Veteran Team Member
- Dr. Beiyu Lin - CS UT Dallas Faculty Advisor
- James Dong - CS Team Member [left after a few minutes of joining]

Note taker:

- Ollie Rayo

Meeting Agenda

- Many team members experienced problems in running risc-v toolchain (created by source)
- Overview of how to build the RISC-V vector toolchain
 - Have to rebuild the proxy kernel
 - Won't work unless you rebuild only with scalar proxy kernel
 - Toolchain isn't made for subset 'rv32imc_zve32x_zvl128b'
 - In test out simulator (limitation)
 - Using 'rv32gcv'
 - Can build compiler with rv32imc_zve32x_zvl128b, run simulator with full vector extensions
 - Only supporting integers (TinyML mostly uses integers)
- Try to replicate results of last semester (this week)
 - Following LiteRT-Vector-Intrinsic-Flow
- Numbers for micro_speech weren't correct or consistent
- Person_detection graph was correct (done by the ECE team)

- We need to decide who is going to be working on what pieces
 - ECE team work on signal processing kernels
 - CS team ai kernels are left to improvise / optimize
 - ECE team will come up with which kernels ECE team and which kernels CS team will be working on
- By this time last year, just gotten toolchain and performance estimator
- Reviewed tflite-micro/tensorflow/lite/micro/examples github
 - Just look at micro_speech -> person_detection (division of tasks, led by ECE team)
- Standard convolution and depth-wise convolution
 - Looked at intrinsic function (to see what team did last semester)
 - Looked at the difference of original and optimized version
 - Standard version: Nested loops, bounds checking (like scalar core)
 - Vector version: trying to vectorize parameters, basically, inner most loop was for 2D convolution and provided the biggest uplift (optimizable)
- Last semester NVIDIA was impressed with results of cs team

Activities agreed upon:

- Need to get into agreement for results (Talk to ECE team)
 - Duplicate the person detection results and micro_speech results
 - Division of tasks (kernels)
- Compare the standard convolution (original) and depth-wise convolution (optimized)
 - conv_rvv.cc
 - See how the intrinsics work

Data and time of the next meeting

- October 24 at 2pm

Name of person taking notes for next meeting

- Avinash Mandala

Deadlines to submit the agenda for the next meeting

- 10/24/2025

15.7 Meeting Minutes 7: 10/24

Project Title: Optimization of LiteRT Micro for RISC-V (Phase 2 - with ECE team)

Date: October 24, 2025

Time: 2:00pm - 2:21pm

Names with Roles:

- Alec Freeman - CS Team Member
- Avinash Mandala - CS Team Member

- James Dong - CS Team Member
- Ollie Rayo - CS Team Member
- Asheesh Kashyap - Company Mentor
- Dr. Beiyu Lin - CS UT Dallas Faculty Advisor
- James Whitfield - ECE Veteran Team Member
- Akhilesh Vemuri - ECE Veteran Team Member
- Jack Liao - ECE Veteran Team Member

Absentees:

- Avinash Mandala - CS Team Member
- James Dong - CS Team Member
- Dr. Beiyu Lin - CS UT Dallas Faculty Advisor

Note taker:

- Ollie Rayo

Meeting Agenda

- Making sure that the CS team was able to replicate the same results as ECE team and last year
 - Person detection and micro speech were the same
- Figuring out what will be coded next week
 - Need to implement AI kernels (8 in total)
 - Members will be assigned to 1 kernel
 - James (ECE team) will delegate kernels and people can volunteer for them, everyone must work on some part to receive credit
- Originally wanted to create a vector processor and vector optimized library to show speed-up, wanted to have that running on an FPGA vector processor
 - However, RISC-V core was not finished
 - But don't have to worry about this part (FPGA vector processor)
- Goal is to be able to say we implemented an AI kernel, ran it on the RISC-V processor, and compare the speed-up
- Akhilesh started a template for the final report and will send out invites later

Activities agreed upon:

- Everyone needs to be assigned a kernel

Data and time of the next meeting

- October 31 at 2pm

Name of person taking notes for next meeting

- Avinash Mandala

Deadlines to submit the agenda for the next meeting

- 10/31/2025

15.8 Meeting Minutes 8: 10/31

Project Title: Optimization of LiteRT Micro for RISC-V (Phase 2 - with ECE team)

Date: October 31, 2025

Time: 2:00pm - 2:11pm

Names with Roles:

- Alec Freeman - CS Team Member
- Avinash Mandala - CS Team Member
- James Dong - CS Team Member
- Ollie Rayo - CS Team Member

- Asheesh Kashyap - Company Mentor
- Dr. Beiyu Lin - CS UT Dallas Faculty Advisor

- James Whitfield - ECE Veteran Team Member
- Akhilesh Vemuri - ECE Veteran Team Member
- Jack Liao - ECE Veteran Team Member

Absentees:

- Dr. Beiyu Lin - CS UT Dallas Faculty Advisor

Note taker:

- Avinash Mandala

Meeting Agenda

- Check on what kernel everyone was assigned to
- Working on TinyML space
 - Dealing with data collection devices.
- Discussed possibility of open-source contribution to TinyML.
- Project plan recap
 - Finish up AI kernels
 - Then move onto signal processing.

Activities agreed upon:

- The original kernel and the optimized kernel
 - So just focus on optimizing the specific kernel you were assigned and present results

Data and time of the next meeting

- November 7 at 2pm

Name of person taking notes for next meeting

- Alec Freeman

Deadlines to submit the agenda for the next meeting

- 11/07/2025

15.9 Meeting Minutes 9: 11/7

Project Title: Optimization of LiteRT Micro for RISC-V (Phase 2 - with ECE team)

Date: November 7, 2025

Time: 2:00pm - 2:30pm

Names with Roles:

- Alec Freeman - CS Team Member
- Avinash Mandala - CS Team Member
- James Dong - CS Team Member
- Ollie Rayo - CS Team Member

- Asheesh Kashyap - Company Mentor
- Dr. Beiyu Lin - CS UT Dallas Faculty Advisor

- James Whitfield - ECE Veteran Team Member
- Akhilesh Vemuri - ECE Veteran Team Member
- Jack Liao - ECE Veteran Team Member

Absentees:

- Dr. Beiyu Lin - CS UT Dallas Faculty Advisor
- James Whitfield - ECE Veteran Team Member
- James Dong - CS Team Member
- Ollie Rayo - CS Team Member

Note taker:

- Alec Freeman

Meeting Agenda

- Round Table : Went around and described what kernels were worked on
- If too easy ask ECE team for more work
- We need to hurry up and get to signal processing
- Don't spend more than another week on AI kernels
- Hasn't been enough work done in general - need to speed up on work done
- AI kernel is just a warm up - we need to optimize a more sophisticated kernel that produces a greater speed up at the application level

Activities agreed upon:

- Single Deliverable: Finish assigned AI kernels
 - Integrate the AI kernels
 - Get the speedup for:
 - Original scalar processor, vector optimizations last semester, vector optimizations this semester so far for the example applications.
 - Have a presentation where you are able to show all of these results

Data and time of the next meeting

- November 14 at 2pm

Name of person taking notes for next meeting

- Avinash Mandala

Deadlines to submit the agenda for the next meeting

- 11/14/2025

15.10 Meeting Minutes 10: 11/14

Project Title: Optimization of LiteRT Micro for RISC-V (Phase 2 - with ECE team)

Date: November 14, 2025

Time: 2:00pm - 2:40pm

Names with Roles:

- Alec Freeman - CS Team Member
- Avinash Mandala - CS Team Member
- James Dong - CS Team Member
- Ollie Rayo - CS Team Member

- Asheesh Kashyap - Company Mentor
- Dr. Beiyu Lin - CS UT Dallas Faculty Advisor

- James Whitfield - ECE Veteran Team Member
- Akhilesh Vemuri - ECE Veteran Team Member
- Jack Liao - ECE Veteran Team Member

Absentees:

- Dr. Beiyu Lin - CS UT Dallas Faculty Advisor
- James Dong - CS Team Member
- Avinash Mandala - CS Team Member
- Akhilesh Vemuri - ECE Veteran Team Member

Note taker:

- Alec Freeman

Meeting Agenda

- Round Table : Went around and described what kernels were worked on and any problems
 - James W. finished MaxPool kernel
- After December 5th - whatever in the main branch is whatever we have as a result

Activities agreed upon:

- Continue working on AI kernels

Data and time of the next meeting

- November 21 at 2pm

Name of person taking notes for next meeting

- Ollie Rayo

Deadlines to submit the agenda for the next meeting

- 11/21/2025

15.11 Meeting Minutes 11: 11/21

Project Title: Optimization of LiteRT Micro for RISC-V (Phase 2 - with ECE team)

Date: November 21, 2025

Time: 2:00pm - 3:00pm

Names with Roles:

- Alec Freeman - CS Team Member
- Avinash Mandala - CS Team Member
- James Dong - CS Team Member
- Ollie Rayo - CS Team Member

- Asheesh Kashyap - Company Mentor
- Dr. Beiyu Lin - CS UT Dallas Faculty Advisor

- James Whitfield - ECE Veteran Team Member
- Akhilesh Vemuri - ECE Veteran Team Member
- Jack Liao - ECE Veteran Team Member

Absentees:

- Dr. Beiyu Lin - CS UT Dallas Faculty Advisor

Note taker:

- Ollie Rayo

Meeting Agenda

- Don't expect a good grade if not showing initiative
- Round Table: Showing current results of working on kernels
 - James W. showed the greatest results with performance speed ups in micro-speech example
- Integration/performance testing for the kernels that are currently passing

Activities agreed upon:

- Get kernels implemented into main branch by Dec 5th
 - Work with James W. and Akhilesh to see performance speed up and see which kernels make it into the final main-branch

Data and time of the next meeting

- December 5th at 2pm

Name of person taking notes for next meeting

- Alec Freeman

Deadlines to submit the agenda for the next meeting

- 12/5/2025

15.12 Meeting Minutes 12: 11/28

Project Title: Optimization of LiteRT Micro for RISC-V (Phase 2 - with ECE team)

Date: November 28, 2025

Names with Roles:

- Alec Freeman - CS Team Member
- Avinash Mandala - CS Team Member
- James Dong - CS Team Member
- Ollie Rayo - CS Team Member

- Asheesh Kashyap - Company Mentor
- Dr. Beiyu Lin - CS UT Dallas Faculty Advisor

- James Whitfield - ECE Veteran Team Member
- Akhilesh Vemuri - ECE Veteran Team Member
- Jack Liao - ECE Veteran Team Member

Meeting Agenda

- No meeting occurred
- Reason: Determined last week that the next meeting was going to be December 5th.

Data and time of the next meeting

- December 5th at 2pm

Name of person taking notes for next meeting

- Alec Freeman

Deadlines to submit the agenda for the next meeting

- 12/5/2025

15.13 Meeting Minutes 13: 12/5

Project Title: Optimization of LiteRT Micro for RISC-V (Phase 2 - with ECE team)

Date: December 5, 2025

Time: 2:00pm - 2:44pm

Names with Roles:

- Alec Freeman - CS Team Member
- Avinash Mandala - CS Team Member
- James Dong - CS Team Member

- Ollie Rayo - CS Team Member
- Asheesh Kashyap - Company Mentor
- Dr. Beiyu Lin - CS UT Dallas Faculty Advisor
- James Whitfield - ECE Veteran Team Member
- Akhilesh Vemuri - ECE Veteran Team Member
- Jack Liao - ECE Veteran Team Member

Absentees:

- Dr. Beiyu Lin - CS UT Dallas Faculty Advisor

Note taker:

- Alec Freeman

Meeting Agenda

- Focus on RISC-V vector intrinsics in presentation
- Anyone can take their libraries and run it on their machine (open source)
- Profiler trying to emulate performance of typical processor implementation
- If someone uses intrinsic libraries these are really helpful
- Current vendors for TinyML: all have implemented some of the functions, but haven't done it for RISC-V libraries.
- Can make contribution depending on confidence that our code works
 - Can contact curator
 - Could independently take source code and try and contribute to actual repository
- Initiative important

Activities agreed upon:

- Integrate
- Presentation on 12/12

Data and time of the next meeting

- This is the last meeting

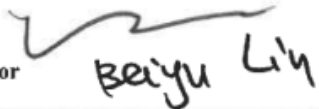
Name of person taking notes for next meeting

- None

Deadlines to submit the agenda for the next meeting

- None

SIGNATURES

 Alec Freeman Date: 12/9/25	 Ollie Rayo Date: 12/9/25
Avinash Mandala  Date: 12/9/25	 James Dong Date: 12/9/25
Date: 12/9/25	 Faculty Advisor Date: 12/9/25

